

AI, ML, and NLP Setup Guide: From Environment to FCA-Ready Explainability

This guide walks you through setting up your Python environment, connecting to Azure Cognitive Services, building your ML pipeline, running NLP analysis, and generating SHAP explanations. Use it alongside the project brief.

Files needed before you start:

- * meridian_customers.csv, meridian_tickets.csv, meridian_reviews.csv, meridian_outcomes.csv
- * Python 3.9 or above with Jupyter Notebook (Anaconda recommended)
- * Optional: a free Microsoft Azure account (portal.azure.com) for Path A NLP

Save all four CSV files to a folder called DACP_AI_Project before opening Jupyter.

PART 1 * Python Environment Setup

Step 1: Install the required Python packages

Open a terminal (Anaconda Prompt on Windows, Terminal on Mac). Run each installation command below. Wait for each to complete before running the next.

```
# Core data science (may already be installed with Anaconda)
pip install pandas numpy matplotlib seaborn scikit-learn scipy

# NLP libraries (Path B - open source)
pip install vaderSentiment
pip install spacy
python -m spacy download en_core_web_sm

# SHAP for explainability (Section F)
pip install shap

# Azure Cognitive Services (Path A - optional)
pip install azure-ai-textanalytics

# Verify installations
python -c "import sklearn, shap, vaderSentiment; print('All installed')"
```

If pip install shap fails with a C++ error on Windows, install it via conda instead: conda install -c conda-forge shap. Conda handles the compiled dependencies automatically.

PART 2 * Azure Cognitive Services Setup (Path A)

Skip this part if you are using Path B (Python open-source NLP). Both paths produce equivalent outputs.

Step 2: Create a free Azure account

Go to azure.microsoft.com and click 'Start for free'. Use a Microsoft account or create one. You will need a credit card to verify your identity but will not be charged on the free tier.

Step 3: Create the Azure AI Language resource

After signing in to portal.azure.com: click Create a resource, search for 'Language', select Azure AI Language. On the Create screen: Subscription = your free subscription. Resource group = create new, name it DACP-AI-Project. Region = UK South. Name = a unique name such as meridian-nlp-yourname. Pricing tier = Free F0. Click Review and Create, then Create.

The Free F0 tier allows 5,000 text analysis transactions per month at no charge. The project uses fewer than 1,000 requests, so you will not exceed this limit. If you accidentally select a paid tier, delete the resource immediately.

Step 4: Copy your endpoint and key

After deployment completes, click Go to resource. In the left menu, click Keys and Endpoint. Copy Key 1 and the Endpoint URL. Keep these safe - do not share them or put them in a public GitHub repository. Paste them into your Jupyter Notebook at the top of Section D.

```
# Add these at the top of your Section D code cell
# Do NOT commit these to GitHub
ENDPOINT = 'https://your-resource-name.cognitiveservices.azure.com/'
KEY       = 'your-32-character-key-here'

# Test the connection
from azure.ai.textanalytics import TextAnalyticsClient
from azure.core.credentials import AzureKeyCredential

client = TextAnalyticsClient(endpoint=ENDPOINT,
                             credential=AzureKeyCredential(KEY))

test = client.analyze_sentiment(documents=['Test sentence.'])
print('Connection successful:', test[0].sentiment)
```

PART 3 * ML Pipeline - Build, Train, and Evaluate

Step 5: Load and merge the structured data

Open Jupyter Notebook and navigate to your DACP_AI_Project folder. Create a new notebook. In the first code cell, load and merge the two structured files. Always check the row count after merging.

Step 6: Handle the class imbalance correctly

Before building any model, check the class balance. The escalation rate is approximately 26.8%, which means the dataset is imbalanced but not severely so. Use `class_weight='balanced'` in both models. This adjusts the training process so the model does not just learn to predict the majority class.

Common mistake:

Without `class_weight='balanced'`, your model may achieve 73% accuracy simply by always predicting 'not escalated'. That is not a useful model. Always check the recall score for the minority class (`escalated = True`). If recall is below 0.50, the model is missing more than half of the real escalations.

Step 7: Evaluate using the business cost framework

After training, do not just report AUC and F1. Run the business cost calculation at three thresholds (0.30, 0.40, 0.50) as shown in Section C. The correct threshold is the one that minimises total cost given the £800 FOS case cost and £25 intervention cost. State your recommended threshold in a markdown cell with the cost comparison.

PART 4 * NLP Analysis

Step 8: Choose your NLP path and stick with it

Decide at the start of Section D which path you will use. Path A (Azure) and Path B (VADER + spaCy) both produce sentiment scores and key phrases. The main difference is that Azure returns named confidence scores per category (positive/neutral/negative as separate floats) whereas VADER returns a single compound score from -1 to +1.

Comparison	Path A - Azure	Path B - Python
Sentiment output	positive, neutral, negative with confidence scores	Compound score -1 to +1, plus pos/neu/neg floats
Key phrases	Built-in <code>extract_key_phrases</code> endpoint	spaCy <code>noun_chunks</code> as proxy
Entity recognition	Built-in <code>recognize_entities</code> endpoint	spaCy <code>ent.label_</code> (ORG, MONEY, DATE, etc.)
Cost	Free tier: 5,000 calls/month	Free, no account required
Setup time	15-20 minutes (Azure portal)	2 minutes (pip install)

Step 9: Process text in batches

Azure has a maximum of 25 documents per API call. For Path B, spaCy can be slow on large datasets. Use batching to avoid timeout errors and slow processing.

```
# Azure: process in batches of 25
def batch_sentiment(texts, batch_size=25):
    results = []
    for i in range(0, len(texts), batch_size):
        batch = texts[i:i+batch_size]
        output = client.analyze_sentiment(documents=batch)
        for doc in output:
            if not doc.is_error:
                results.append({
                    'sentiment': doc.sentiment,
                    'positive': doc.confidence_scores.positive,
                    'negative': doc.confidence_scores.negative,
                })
    return pd.DataFrame(results)
```

```
# Path B: limit spaCy to 200-300 texts first
# Full dataset processing can take 10-15 minutes
sample_texts = tickets['ticket_text'].head(200)
```

Step 10: Produce the five required NLP outputs

After running sentiment analysis and key phrase extraction, build all five outputs listed in Section D of the project brief. Create each chart in a separate code cell with a markdown cell above stating what you are looking for and a markdown cell below with your finding. Do not generate all five charts in one code cell.

PART 5 * SHAP Explainability (Section F)

Step 11: Install and verify SHAP before you need it

Run `pip install shap` in your terminal before you reach Section F. SHAP has compiled dependencies that can take several minutes to install. If the installation fails, try `conda install -c conda-forge shap` instead.

Step 12: Use the correct SHAP explainer for your model

TreeExplainer works with Random Forest, XGBoost, and other tree-based models. LinearExplainer works with Logistic Regression, Ridge, and Lasso. Using the wrong explainer will not necessarily produce an error, but the SHAP values will be inaccurate.

```
import shap

# Check which model you trained
print(type(chosen_model))

# Random Forest: use TreeExplainer
# from sklearn.ensemble import RandomForestClassifier
# -> shap.TreeExplainer(model)

# Logistic Regression: use LinearExplainer
# from sklearn.linear_model import LogisticRegression
# -> shap.LinearExplainer(model, background_data)

# The background data for LinearExplainer
# should be your TRAINING set, not the test set
# explainer = shap.LinearExplainer(lr_model, X_train_scaled)
```

Step 13: Generate and read the global SHAP summary chart

The global summary bar chart shows average absolute SHAP values per feature. A higher bar means that feature contributes more to the model's predictions on average. This is NOT the same as a positive correlation with escalation - a feature can be important even if it both increases and decreases risk depending on its value.

Step 14: Write the plain-English regulator explanations

After generating the three waterfall charts (True Positive, False Positive, False Negative), write a plain-English explanation for each. Read the waterfall chart from bottom to top - the bottom bars are the

features that pushed the prediction up or down the most. Use the template in Section F of the project brief.

How to read a SHAP waterfall chart:

The starting point ($E[f(x)]$) is the model's average prediction across all customers. Each row is a feature. Red bars push the prediction higher (towards escalated). Blue bars push it lower (towards not escalated). The final bar at the top is the individual customer's predicted probability. A regulator explanation should say: 'The three factors that contributed most to this customer being flagged were X (increased risk by Y%), Z (increased risk by W%), and Q (reduced risk by V%).'

PART 6 * Power BI Integration (Section G)

Step 15: Export the ML and NLP outputs as CSVs

Run the export code in Section G of the brief before opening Power BI. You should have two output files: `meridian_ml_output.csv` (customer risk scores and bands) and `meridian_nlp_output.csv` (ticket text with sentiment scores). Verify both files open correctly in Excel before loading them into Power BI.

Step 16: Load both CSVs into Power BI and create the relationship

In Power BI, load both CSVs. The model will automatically detect that both files have a `customer_id` column. Create a Many-to-One relationship from `meridian_nlp_output[customer_id]` to `meridian_ml_output[customer_id]`. This allows you to show a customer's sentiment trend alongside their ML risk score on the same page.

Step 17: Add the Key Influencers visual to Page 1

On Page 1 (Escalation Risk Overview), add the Key Influencers visual from the Visualisations pane. Drag `risk_band` to the Analyse well and drag `product_type`, `age_band`, `tenure_months`, and `previous_complaints` to the Explain by well. The visual will automatically identify which factors most increase the probability of a customer being in the High Risk band. Screenshot this for your submission.

Final check before submitting:

- * Your notebook runs from top to bottom without errors after a Kernel > Restart and Run All
- * All five NLP output charts are present with markdown observations
- * Three SHAP waterfall charts generated and plain-English explanations written
- * Power BI dashboard has all four pages and at least one AI-native visual
- * Responsible AI Summary is one page, written for a non-technical compliance reader